UNIVERSITÉ DE MOUNDOU — TCHAD

Master en Informatique — Data Science et Intelligence Artificielle

RAPPORT DE TP D'EXAMEN — PARTIE 2

Gestion d'une ligne de commande

Application mobile (Flutter)

Réalisé par : Mahamat Issa et Ibrahim Adam Fadoul

Flutter · Dart · API REST Laravel · MySQL

12 octobre 2026

Sommaire

1. Introduction et objectifs
2. Contexte : sujet et données partagées
3. Architecture et environnement de développement
4. L'API REST côté Laravel
5. Structure de l'application Flutter
6. Écrans et fonctionnalités
7. Gestion des erreurs et ergonomie
8. Exécution et déploiement
9. Validation et scénarios de test
10. Difficultés rencontrées et solutions
11. Limites et perspectives
12. Conclusion

Annexe — Extraits de code

1. Introduction et objectifs

Ce rapport présente la partie mobile du TP d'examen « Gestion d'une ligne de commande ». Après la réalisation de l'application web avec Laravel (premier rapport), l'objectif est de rendre les mêmes fonctionnalités disponibles sur téléphone, avec le framework Flutter.

L'application mobile ne se connecte pas directement à la base de données : elle dialogue avec une API REST ajoutée au projet Laravel, qui renvoie des données au format JSON. Ce choix présente plusieurs avantages :

- les règles de gestion (contrôle et mise à jour du stock, transactions, validation) restent centralisées côté serveur et ne sont pas dupliquées ;
- le site web et l'application mobile travaillent sur la même base : une modification faite sur l'un est immédiatement visible sur l'autre ;
- aucun identifiant de base de données n'est embarqué dans l'application mobile.

Les objectifs sont de reprendre les cinq travaux de l'énoncé dans une interface adaptée au mobile, de gérer proprement les erreurs réseau et de décrire la mise en œuvre sur un appareil réel.

2. Contexte : sujet et données partagées

2.1 Les cinq travaux et leur déclinaison mobile

N°	Travail demandé	Réalisation mobile
1	Formulaire d'enregistrement des produits d'une commande	Écran « Nouvelle commande » : client, date, lignes produit/quantité, total estimé.
2	Afficher les listes des produits, des commandes et des clients	Onglets Produits, Commandes et Clients.
3	Modifier un produit	Appui sur un produit : formulaire de modification.
4	Supprimer un produit dans une commande	Détail d'une commande : icône corbeille sur chaque ligne, avec confirmation.
5	Tableau de bord statistique	Onglet Tableau : indicateurs, top 5, CA par région et par mois, alertes de stock.

2.2 Base de données partagée

La base MySQL tp_commande suit le schéma relationnel de l'énoncé : Departement, Region, Client, Produit, Commande et ligneCommande (clé primaire composée idCmd + idProd). Elle est décrite en détail dans le premier rapport. L'application mobile lit et modifie ces mêmes tables par l'intermédiaire de l'API. La capture ci-dessous montre la table produits dans phpMyAdmin : ce sont les données que l'API envoie à l'application.

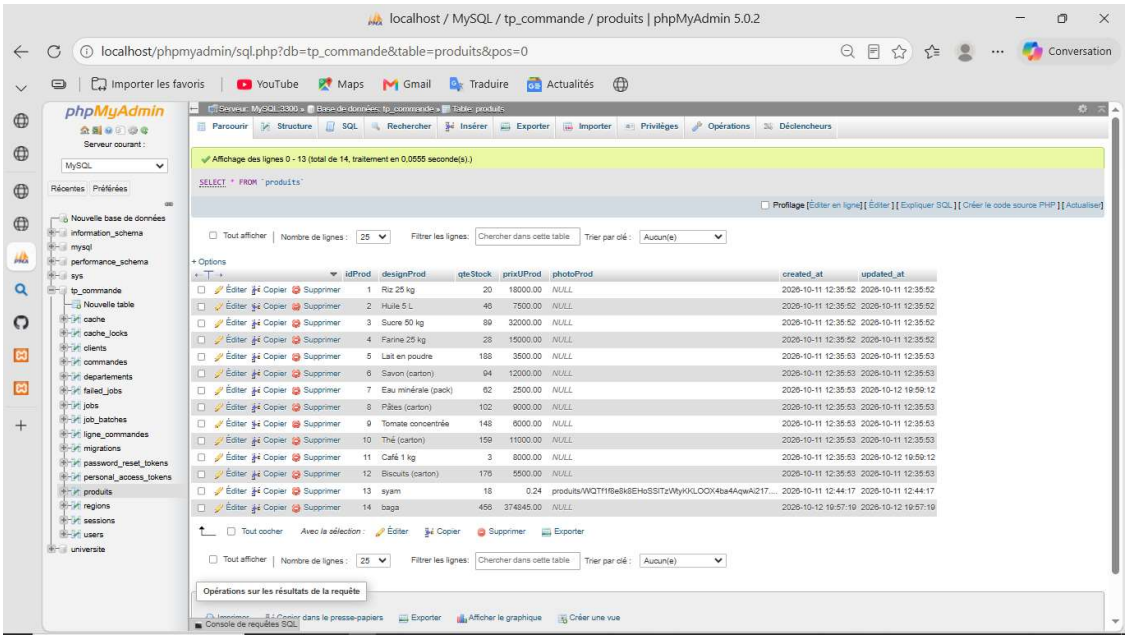


Figure 1 — phpMyAdmin — table produits, source des données affichées sur mobile

3. Architecture et environnement de développement

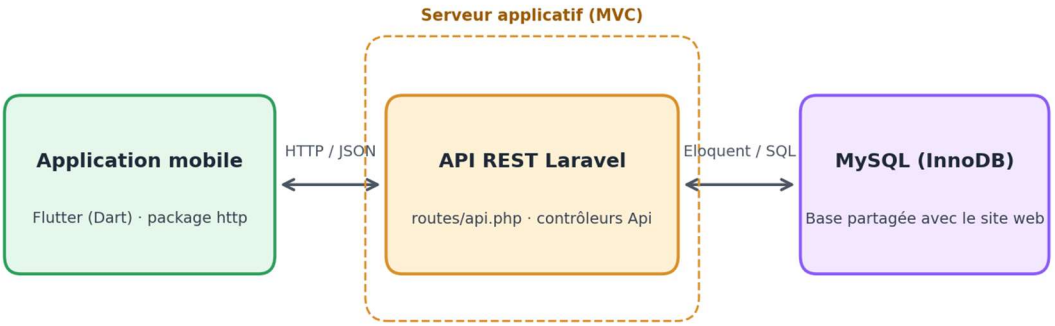


Figure 2 — Architecture de l'application mobile

Élément	Technologie	Rôle
Application mobile	Flutter, langage Dart, Material 3	Interface utilisateur multiplateforme (Android, iOS, web).
Communication	Package http, format JSON	Appels GET, POST, PUT et DELETE vers l'API.
API	Laravel 12, routes/api.php	Expose les données et applique les règles de gestion.
Base de données	MySQL (InnoDB)	Stockage, partagé avec l'application web.

Environnement utilisé

- Flutter SDK et Dart, avec la commande flutter doctor pour contrôler l'installation ;
- Android Studio, SDK Android et outils en ligne de commande (cmdline-tools) pour compiler pour Android ;
- Chrome, qui permet de tester rapidement l'application en mode « appareil mobile » sans émulateur ;
- PHP 8.3, Laravel 12 et MySQL (Laragon) pour le serveur de l'API.

4. L'API REST côté Laravel

4.1 Mise en place

L'API est ajoutée au projet web existant avec la commande php artisan install:api, qui crée le fichier routes/api.php (toutes les routes sont préfixées par /api). Quatre contrôleurs, regroupés dans le dossier app/Http/Controllers/Api, réutilisent les modèles Eloquent du projet : ProduitApiController, ClientApiController, CommandeApiController et DashboardApiController. Les réponses sont mises en forme explicitement (types entiers et décimaux garantis) pour que l'application Flutter n'ait pas à deviner les formats.

4.2 Routes disponibles

Méthode	Route	Rôle
GET	/api/dashboard	Indicateurs, top 5 produits, CA par région et par mois, alertes de stock
GET	/api/produits	Liste des produits
POST	/api/produits	Création d'un produit
PUT	/api/produits/{id}	Modification d'un produit
GET	/api/clients	Liste des clients avec région et département
GET	/api/commandes	Liste des commandes (client, nombre de produits, total)
GET	/api/commandes/{id}	Détail d'une commande et de ses lignes
POST	/api/commandes	Création d'une commande (transaction et contrôle du stock)
DELETE	/api/commandes/{id}/produits/{idProd}	Retrait d'un produit d'une commande, avec restitution du stock

4.3 Format des échanges

Les données circulent en JSON. Les exemples suivants montrent la réponse du tableau de bord, le corps d'une création de commande et une réponse d'erreur de validation (code HTTP 422), dont le message est affiché tel quel à l'utilisateur :

```
GET /api/dashboard
{
  "kpi": { "ca": 6803500, "commandes": 80, "panier_moyen": 85043.75, "unites": 614 },
  "top_produits": [ { "label": "Sucre 50 kg", "ca": 1504000 }, ... ],
  "par_region":   [ { "label": "Sarh", "ca": 1364000 }, ... ],
  "par_mois":     [ { "label": "2026-03", "ca": 977500 }, ... ],
  "stock_faible": [ { "idProd": 11, "designProd": "Café 1 kg", "qteStock": 3 } ]
}

POST /api/commandes   (corps de la requête)
{ "idCli": 5, "dateCmd": "2026-10-12",
  "lignes": [ { "idProd": 2, "qteCmd": 3 }, { "idProd": 7, "qteCmd": 10 } ] }

Réponse d'erreur (code 422)
{ "message": "Stock insuffisant pour « Riz 25 kg » (disponible : 20).",
  "errors": { "lignes": [ "Stock insuffisant pour « Riz 25 kg » (disponible : 20)." ] } }
```

4.4 Règles de gestion appliquées côté serveur

- création de commande dans une transaction avec verrouillage des produits : si le stock est insuffisant, toute la commande est annulée ;
- suppression d’une ligne de commande avec restitution de la quantité au stock, dans une transaction ;
- validation de toutes les entrées (client existant, quantités entières positives, produits distincts, prix numérique positif) ;
- erreurs renvoyées en JSON avec le bon code HTTP (422 pour une donnée invalide, 404 pour une ressource inconnue).

5. Structure de l'application Flutter

5.1 Organisation des fichiers

Fichier	Rôle
main.dart	Point d'entrée, thème Material 3 et navigation par onglets (Tableau, Produits, Commandes, Clients).
config.dart	Adresse du serveur (baseUrl), à adapter selon l'environnement.
api.dart	Client HTTP : requêtes GET/POST/PUT/DELETE, décodage JSON, lecture des erreurs de validation, délai maximal de 15 secondes.
utils.dart	Formatage des montants (FCFA) et des dates, widgets réutilisables (Chargement, BarList, Carte).
pages/dashboard_page.dart	Tableau de bord.
pages/produits_page.dart	Liste des produits et formulaire de création/modification.
pages/commandes_page.dart	Liste, détail (suppression d'une ligne) et création d'une commande.
pages/clients_page.dart	Liste des clients.

5.2 Principes de conception

- **Navigation :** une barre d'onglets inférieure (NavigationBar) donne accès aux quatre sections ; chaque onglet est recréé à l'ouverture, ce qui garantit des données à jour.
- **Chargement des données :** chaque page lance un appel à l'API (Future) et l'affiche avec un FutureBuilder, encapsulé dans un widget générique « Chargement » qui gère l'attente, l'erreur et le contenu.
- **Client HTTP :** toutes les requêtes passent par la classe Api, qui ajoute les en-têtes JSON, applique un délai maximal et transforme les erreurs du serveur en exceptions lisibles (ApiException).
- **Réutilisation :** les barres de statistiques, les cartes et le formatage des montants sont écrits une seule fois et partagés par tous les écrans.
- **Choix de visualisation :** les statistiques sont représentées par des barres horizontales proportionnelles, plus lisibles sur un petit écran qu'un graphique classique et sans dépendance supplémentaire.

Les extraits de code principaux sont donnés en annexe (A.2 et A.3).

6. Écrans et fonctionnalités

Remarque : les captures ont été réalisées en exécutant l'application dans Chrome en mode « appareil mobile ». Le même code s'installe sur un téléphone Android (voir section 8).

6.1 Tableau de bord (travail 5)

L'écran affiche quatre indicateurs (chiffre d'affaires 6 803 500 FCFA, 80 commandes, panier moyen 85 044 FCFA, 614 unités vendues), puis le top 5 des produits, le chiffre d'affaires par région et l'évolution mensuelle. Ces valeurs sont identiques à celles du tableau de bord web, puisqu'elles viennent des mêmes requêtes SQL. Un geste de tirage vers le bas rafraîchit les données.

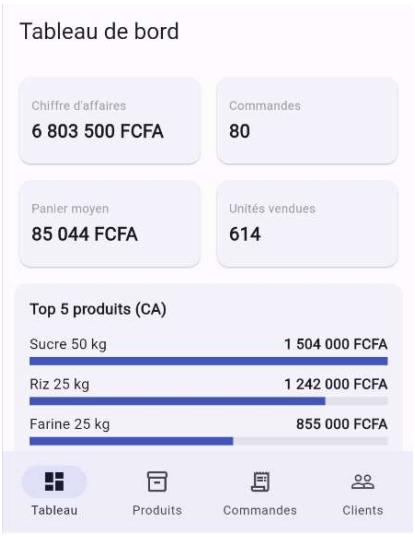


Figure 3 — Indicateurs et top produits



Figure 4 — CA par région



Figure 5 — Évolution mensuelle

6.2 Produits (travaux 2 et 3)

La liste affiche la désignation, le stock (en rouge sous le seuil de 10 unités) et le prix. Un appui sur un produit ouvre le formulaire de modification ; le bouton « + » ouvre le même formulaire pour une création. Les champs sont contrôlés avant l'envoi (champ obligatoire, entier positif, prix positif) et les erreurs renvoyées par le serveur sont affichées dans une barre de message.

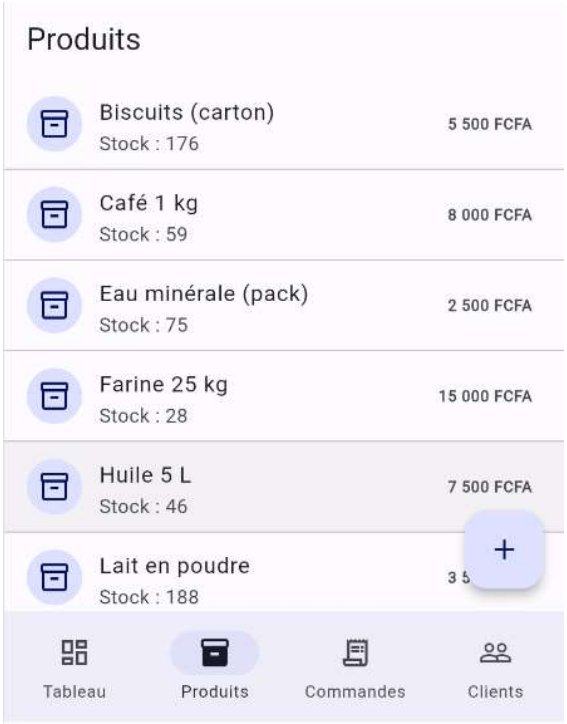


Figure 6 — Liste des produits

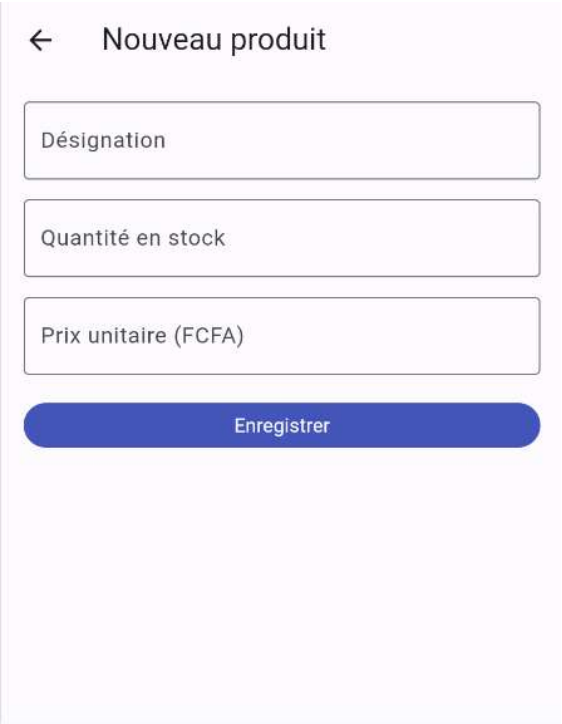


Figure 7 — Formulaire de produit

6.3 Commandes

La liste des commandes présente, pour chaque commande, le client, la date, le nombre de produits et le total. Un appui ouvre le détail, où chaque ligne peut être supprimée après confirmation : le stock est alors restitué par le serveur (travail 4).

L'écran « Nouvelle commande » (travail 1) propose la sélection du client, un sélecteur de date, des lignes produit/quantité que l'on peut ajouter ou retirer, et un total estimé recalculé en temps réel. Seuls les produits en stock sont proposés ; si la quantité dépasse le stock, le serveur refuse la commande et le message d'erreur s'affiche.

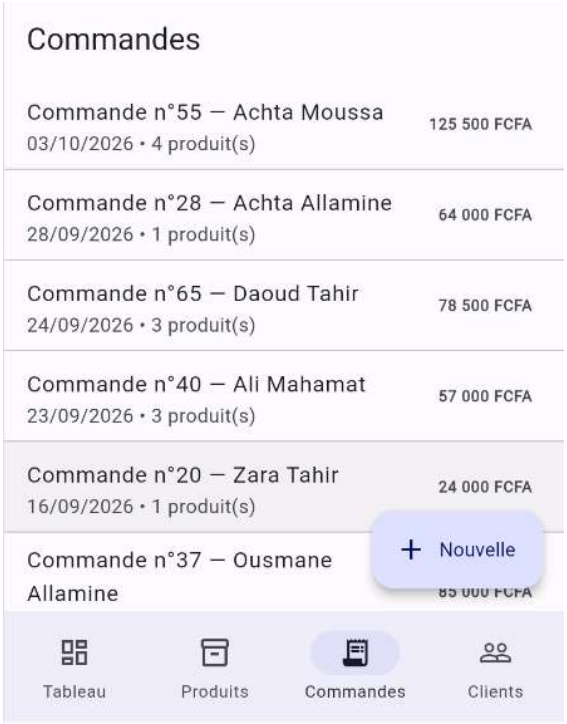


Figure 8 — Liste des commandes



Figure 9 — Nouvelle commande

6.4 Clients

La liste affiche l'initiale, le nom complet, l'email, la région et le département de chaque client.

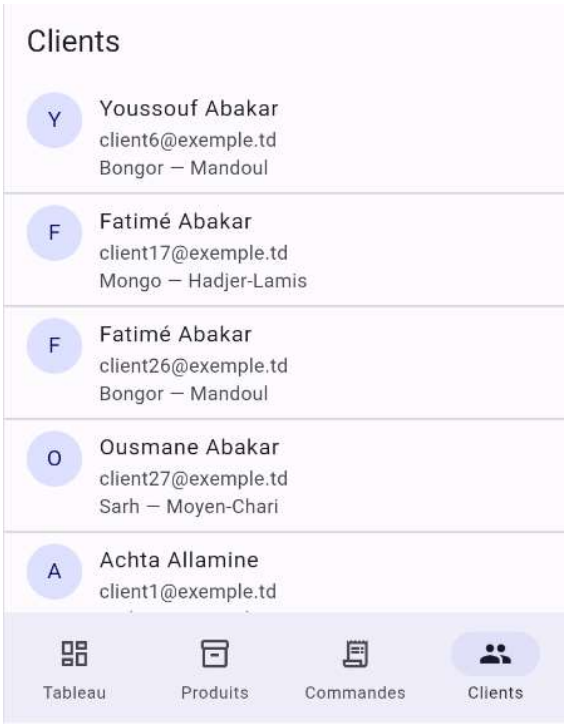


Figure 10 — Liste des clients

7. Gestion des erreurs et ergonomie

- chaque écran affiche un indicateur de chargement, puis un message clair avec un bouton « Réessayer » si le serveur est injoignable ;
- les erreurs de validation du serveur (par exemple « Stock insuffisant ») sont affichées telles quelles à l'utilisateur ;
- un délai maximal de 15 secondes évite qu'un écran reste bloqué en attente ;
- les listes se rafraîchissent par un geste de tirage, et les onglets rechargent leurs données à chaque ouverture ;
- les boutons d'enregistrement sont désactivés pendant l'envoi pour éviter les doubles soumissions ;
- les montants sont formatés en FCFA avec séparateur de milliers, et les dates au format jj/mm/aaaa.

La capture suivante montre le comportement lorsque l'application ne parvient pas à joindre le serveur (adresse incorrecte dans config.dart) : le message indique la cause probable et le bouton « Réessayer » permet de relancer la requête sans quitter l'écran.

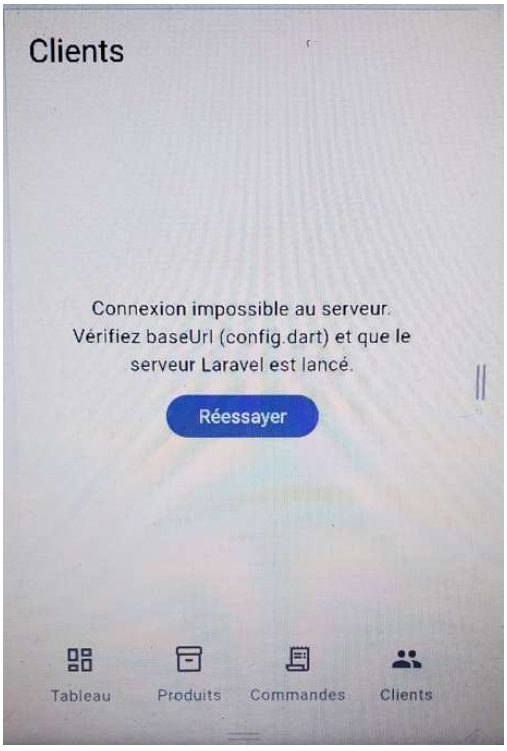


Figure 11 — Message d'erreur affiché lorsque le serveur est injoignable

8. Exécution et déploiement

8.1 Adresse du serveur selon l'environnement

L'adresse de l'API est définie dans une seule constante (baseUrl du fichier config.dart). Elle dépend de l'endroit où l'application s'exécute :

Environnement	Valeur de baseUrl	Remarque
Chrome (test rapide)	http://127.0.0.1:8000/api	Aucun émulateur nécessaire ; commande flutter run -d chrome.
Émulateur Android	http://10.0.2.2:8000/api	10.0.2.2 désigne le PC hôte vu depuis l'émulateur.
Téléphone réel (Wi-Fi)	http://IP_DU_PC:8000/api	IPv4 donnée par ipconfig ; PC et téléphone sur le même réseau.

8.2 Serveur Laravel

Pour qu'un téléphone puisse joindre l'API, le serveur doit écouter sur toutes les interfaces : php artisan serve --host=0.0.0.0 --port=8000. Le pare-feu Windows doit autoriser PHP sur le réseau privé. Un test simple consiste à ouvrir http://IP_DU_PC:8000/api/clients dans le navigateur du téléphone : une liste JSON doit s'afficher.

8.3 Configuration Android

Android bloque par défaut le trafic non chiffré et exige une permission réseau. En développement (HTTP), il faut modifier le fichier AndroidManifest.xml (extrait en annexe A.4). En production, on utiliserait HTTPS.

8.4 Lancement et installation

- **Test dans Chrome :** flutter run -d chrome.
- **Téléphone en USB :** activer le débogage USB, vérifier avec flutter devices, puis flutter run. La commande adb reverse tcp:8000 tcp:8000 permet d'utiliser 127.0.0.1 comme adresse du serveur via le câble.
- **Fichier APK :** flutter build apk --release produit build/app/outputs/flutter-apk/app-release.apk, à copier puis installer sur le téléphone.

9. Validation et scénarios de test

Les premiers scénarios sont établis par les captures du rapport ; les suivants correspondent aux règles implémentées dans le code de l'application et de l'API.

Scénario	Comportement observé ou attendu	Preuve / emplacement
Affichage des listes à partir de l'API	Produits, commandes et clients s'affichent avec leurs données (stock, total, région, département).	Figures 6, 8 et 10
Cohérence avec l'application web	Mêmes indicateurs que le tableau de bord web : 6 803 500 FCFA, 80 commandes, 85 044 FCFA, 614 unités.	Figures 3 à 5
Formulaire de commande	Sélection du client, date, lignes dynamiques et total estimé en temps réel.	Figure 9
Formulaire de produit	Trois champs contrôlés avant envoi ; enregistrement par POST ou PUT.	Figure 7
Serveur injoignable	Message d'erreur explicite et bouton « Réessayer ».	Figure 11
Quantité supérieure au stock	Le serveur répond 422 ; le message « Stock insuffisant » est affiché et rien n'est enregistré.	CommandeApiController@store, api.dart
Suppression d'un produit dans une commande	Confirmation, appel DELETE, puis actualisation du détail ; le stock est restitué par le serveur.	commandes_page.dart, CommandeApiController@retirerProduit
Cohérence des données	Les données de la base (produits, stocks) sont celles envoyées par l'API.	Figure 1

10. Difficultés rencontrées et solutions

Difficulté	Cause	Solution
Commande « flutter » non reconnue	Flutter non installé ou dossier bin absent du PATH.	Installation du SDK dans un dossier sans espace, ajout de flutter\bin au PATH, vérification par flutter doctor.
« Android sdkmanager not found »	Outils en ligne de commande Android (cmdline-tools) non installés.	Installation via le SDK Manager d'Android Studio, flutter config --android-sdk, puis acceptation des licences.
Copie des fichiers impossible (chemin introuvable)	Archive du projet non extraite avant la copie.	Extraction préalable de l'archive, puis copie avec des chemins complets.
API absente (erreur 404 sur /api/…)	Fichier routes/api.php non installé.	php artisan install:api avant la copie des contrôleurs API.
« Connexion impossible au serveur »	baseUrl en 10.0.2.2, adresse réservée à l'émulateur Android, inutilisable dans Chrome.	baseUrl adaptée à l'environnement (127.0.0.1 pour Chrome), puis redémarrage complet de l'application.
HTTP bloqué sur Android	Android interdit par défaut le trafic non chiffré.	Permission INTERNET et attribut usesCleartextTraffic dans AndroidManifest.xml (développement).

11. Limites et perspectives

- **Sécurité :** l'API n'est pas authentifiée. Une mise en production nécessiterait une authentification (Laravel Sanctum, jeton stocké de façon sécurisée sur le téléphone), des rôles et HTTPS.
- **Photos :** l'envoi de photos de produits et de clients, géré sur le web, n'est pas repris sur mobile (appareil photo et envoi de fichier à prévoir).
- **Volumes :** les listes de l'API ne sont pas paginées (la liste des commandes est limitée aux 200 plus récentes) ; une pagination et un défilement infini seraient nécessaires avec de gros volumes.
- **Mode hors ligne :** l'application nécessite le serveur ; un cache local permettrait de consulter les dernières données sans réseau.
- **Statistiques :** graphiques interactifs (bibliothèque dédiée) et filtres par période.
- **Qualité :** tests automatisés (tests de widgets Flutter, tests de l'API) et essais sur plusieurs appareils réels.

12. Conclusion

L'application Flutter reprend les cinq travaux du TP sur mobile : formulaire de commande, listes, modification d'un produit, suppression d'un produit dans une commande et tableau de bord. Elle s'appuie sur une API REST ajoutée au projet Laravel, ce qui garde la logique métier au même endroit et assure la cohérence avec l'application web, puisque les deux partagent la même base.

Le travail a permis de mettre en pratique la séparation entre interface et serveur, la gestion des erreurs réseau et de validation, et la configuration d'un environnement mobile (SDK, adresses réseau, permissions Android). Les évolutions prioritaires sont l'authentification, la gestion des photos et la pagination.

Annexe — Extraits de code

A.1 Routes de l'API (routes/api.php)

```
Route::get('dashboard', [DashboardApiController::class, 'index']);


Route::get('produits', [ProduitApiController::class, 'index']);
Route::post('produits', [ProduitApiController::class, 'store']);
Route::put('produits/{produit}', [ProduitApiController::class, 'update']);


Route::get('clients', [ClientApiController::class, 'index']);


Route::get('commandes', [CommandeApiController::class, 'index']);
Route::get('commandes/{commande}', [CommandeApiController::class, 'show']);
Route::post('commandes', [CommandeApiController::class, 'store']);
Route::delete('commandes/{commande}/produits/{produit}', [CommandeApiController::class,
'retirerProduit']);
```

A.2 Client HTTP (lib/api.dart) : décodage des réponses et gestion des erreurs

```
static dynamic _decode(http.Response r) {
  dynamic body;
  try { body = r.body.isEmpty ? null : jsonDecode(r.body); } catch (_) { body = null; }
  if (r.statusCode >= 200 && r.statusCode < 300) return body;


  // Erreurs de validation Laravel (422) : on affiche tous les messages
  if (body is Map && body['errors'] is Map) {
    final msgs = (body['errors'] as Map).values
        .expand((v) => v is List ? v : [v]).join('\n');
    throw ApiException(msgs);
  }
  throw ApiException('Erreur serveur (${r.statusCode})');
}


static Future<dynamic> _send(Future<http.Response> Function() call) async {
  http.Response r;
  try {
    r = await call().timeout(const Duration(seconds: 15));
  } catch (_) {
    throw ApiException('Connexion impossible au serveur.');
  }
  return _decode(r);
}
```

A.3 Enregistrement d'une commande (lib/pages/commandes_page.dart)

```
Future<void> _save() async {
  if (_idCli == null) { snack(context, 'Choisissez un client.'); return; }
  final lignes = <Map<String, dynamic>>[];
  for (final l in _lignes) {
    final q = int.tryParse(l.qte.text.trim()) ?? 0;
    if (l.idProd == null || q < 1) {
      snack(context, 'Complétez chaque ligne : produit et quantité (au moins 1).');
      return;
    }
    lignes.add({'idProd': l.idProd, 'qteCmd': q});
  }
  setState(() => _busy = true);
  try {
    await Api.post('/commandes', {'idCli': _idCli, 'dateCmd': _dateIso, 'lignes': lignes});
    if (!mounted) return;
    Navigator.pop(context, true);
  } on ApiException catch (e) {      // ex. « Stock insuffisant pour … »
    if (!mounted) return;
    setState(() => _busy = false);
    snack(context, e.message);
  }
}
```

A.4 Configuration Android (AndroidManifest.xml)

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-permission android:name="android.permission.INTERNET"/>

<application
    android:label="commandes_mobile"
    android:usesCleartextTraffic="true"   <!-- HTTP en développement -->
    ...>
```